

LangChain Runnable 与链式编排

DOC_ID: DOC-LC-001

说明：本文档为 RAGLab 项目生成的教学用合成语料，用于分块、检索、重排、引用和评测实验。它不是 LangChain/LangGraph 官方文档，也不应被当作 API 版本说明。文中的稳定标识（DOC_ID、SECTION_ID）用于构建可重复的检索评测集。

1. Runnable 的统一抽象

SECTION_ID: LC-RUN-01

Runnable 可以被理解为一类具有统一执行接口的计算单元。它接收输入，完成转换，再返回输出。模型、提示模板、检索器、输出解析器以及普通 Python 函数，都可以被包装成可组合的执行节点。

在工程上，统一抽象的主要价值不是减少代码行数，而是让不同环节可以用相似的方式执行、替换和测试。一个标准的 RAG 流程可以表示为：

```
用户问题 -> 查询预处理 -> Retriever -> 上下文格式化 -> Prompt -> LLM -> 输出解析
```

每个箭头两侧都应明确输入和输出类型。链路出现错误时，首先检查的不是模型能力，而是前一节点输出是否符合后一节点的预期。

2. 顺序、并行与透传

SECTION_ID: LC-RUN-02

顺序组合表示前一个节点的输出直接进入下一个节点。并行组合表示同一份输入被送入多个分支，分支结果再汇总为一个字典。透传节点用于在转换过程中保留原始问题。

典型 RAG 输入需要同时构造 `question` 和 `context`：

```
inputs = {
    "question": original_question,
    "context": retrieved_documents,
}
```

其中 `question` 可以直接透传，`context` 则来自检索分支。并行结构的价值在于显式展示数据来源，避免把查询、历史和检索结果混在一个不可追踪的字符串中。

3. invoke、batch 与 stream

SECTION_ID: LC-RUN-03

`invoke` 适合执行单条输入，`batch` 适合批量离线评测，`stream` 适合前端逐步展示。三种模式对应不同的工程目标：

模式	主要用途	常见风险
invoke	单次请求、调试	忽略吞吐和并发
batch	测试集、批量生成	需要控制并发和费用
stream	用户界面、长回答	前端需处理增量事件和中断

批量评测时，应为每条样本保留独立的运行标识，防止不同样本共享对话历史或缓存状态。

4. 结构化输出

SECTION_ID: LC-RUN-04

结构化输出的目的不是让文本看起来整齐，而是让下游程序能够稳定消费模型结果。例如 RAG 回答可以使用如下结构：

```
{
  "answer": "...",
  "citations": [
    {"doc_id": "DOC-LG-001", "section_id": "LG-CP-01"}
  ],
  "confidence": "medium",
  "insufficient_evidence": false
}
```

当模型输出不符合结构时，应通过验证错误触发重试、降级或拒绝，而不是在业务代码里用大量字符串切割勉强解析。

5. 配置、回调与可观测性

SECTION_ID: LC-RUN-05

链的配置可以承载运行名称、标签、元数据、超时和回调。调试 RAG 时，应至少记录：原始查询、改写后的查询、召回文档 ID、排序分数、最终上下文、模型输出、耗时和 Token 用量。

只记录最终答案会导致问题无法定位。例如答案错误可能来自：

1. 查询改写偏离原意；
2. Retriever 没有召回正确文档；
3. Reranker 把正确文档排到后面；
4. 上下文截断丢失关键句；
5. 模型没有忠实使用上下文。

因此，Runnable 链路应被视为可观察的数据管道，而不是一个黑盒函数。